Packaging the Google Fonts library as Debian packages — a self-hosted validation & distribution plan

Model: Claude Opus 4.8 (1M context) · drafted 2026-06-05 · **Status: proposed plan**

0. Premise & purpose

Decision (Felipe, 2026-06-05): build and **self-maintain** Debian binary packages for the **entire Google Fonts library** (2,000+ families), published in a **complementary apt repository** — explicitly **not** seeking official Debian inclusion. Rationale:

> A package that builds cleanly, in an isolated chroot, **from our recorded recipe** is *executable proof* that our build-step documentation — the per-family build manifests of `build-fix-provenance.md` — is correct and complete.

Distribution to interested users is a welcome by-product; the **primary goal is verification at scale**. This reframes Debian packaging from "distribution channel" (which I previously argued against) into **the verification harness for the build-fix-provenance manifests** — which is genuinely valuable and squarely on-mission.

1. Why this is the right verification harness

- A source package built with **`sbuild` in a minimal, network-isolated chroot** succeeds only if **every** build dependency is declared and present. That is exactly what our manifests assert: system packages (B), the resolved toolchain (A), pre-build steps (C), source+commit (D), compiler (E). **A green `sbuild` ⇒ the manifest is sufficient; a red `sbuild` ⇒ a concrete, named gap** that feeds straight back into the manifest. The package farm becomes the **enforcement mechanism for manifest completeness** — the §7 "round-trip reproduction in a clean environment" of the provenance plan, implemented with battle-tested tooling.
- `debian/rules` + `sbuild` is, in effect, a **concrete instance of the "tiny runner"** we specified: a standalone consumer of the manifest that reconstructs the environment and builds. One mechanism gives us the runner *and* the clean-room verifier *and* apt distribution.

2. Scope & non-goals

In scope: 2,000+ binary font packages, built **from upstream source via our recipe** (not repacked prebuilt TTFs — repacking would prove nothing about build steps); the **build-tool packages** they depend on (§5); all published to a signed apt repo we host and maintain.

Non-goals: official Debian / NEW-queue submission; Debian Policy *purity*. We deliberately relax the rules that only matter for official archive membership (e.g. "every build-dep must already be in the official archive") while **keeping every rule that gives us proof** — clean chroot, fully declared dependencies, reproducible offline builds.

3. Package model (font packages)

- **Source package = one upstream repo @ pinned commit** (the provenance unit). Many GF families share a repo, so source-package count < family count.
- **Binary package = one per family**, named `fonts-gf-<family-slug>`, `Architecture: all` (fonts are arch-independent → no per-arch build matrix), `Section: fonts`, installing to `/usr/share/fonts/{truetype,opentype}/gf-<family>/`.
- **No file clashes with official Debian font packages:** distinct `fonts-gf-*` namespace + `gf-<family>` install subdir. Where we overlap an official package (e.g. our `fonts-gf-roboto` vs Debian's `fonts-roboto`) we never claim the same paths; users opt into our repo knowingly.

4. Generating `debian/` from the manifest (the linchpin)

A generator turns each family's build manifest into a `debian/` tree — **no per-family hand-authoring**:

- `debian/control` — `Source`, `Maintainer` (us), `Build-Depends` = `system_packages` (B)
- the build-tool packages (§5) + `debhelper-compat`; one `Package: fonts-gf-<family>` stanza per family, `Architecture: all`, `Depends: ${misc:Depends}`, `Homepage`, description from METADATA.
- `debian/rules` — `dh $@` with `override_dh_auto_build` running our recipe: stage the pinned source, run `pre_build` steps (C), invoke `gftools-builder` / `fontc` (E) with the family `config.yaml` (D); `override_dh_auto_install` placing the built fonts.
- `debian/copyright` — **DEP-5**, generated from the family license (OFL-1.1 / Apache-2.0 / UFL-1.0) + `Source:` upstream URL + holders parsed from `OFL.txt`. *(Realizes the DEP-5 adoption from the provenance doc's §9.)*
- `debian/changelog` — version `<upstream_version>+gf<YYYYMMDD>.g<shortcommit>-1`, encoding the **exact upstream commit** so apt upgrades track GF updates and provenance is legible in the version string.
- `debian/watch` — upstream repo + version detection (drives the maintenance loop, §8).
- `debian/patches/` — source-modifying fixes (e.g. the `ofl/moiraione` filename-case fix) as **DEP-3-tagged** quilt patches; build-time-only fixes stay in `rules`. *(Realizes DEP-3.)*
- `debian/source/format` → `3.0 (quilt)`; `debian/upstream/metadata` → repo, commit, provenance link.

The generator is a **pure function of the manifest** — same source of truth, two consumers (`run_manifest` and the Debian packaging). Best built as an `--export deb` **mode in** `gflib-build`, reusing its discovery, cohort, and manifest machinery.

5. Build-dependency tool packages (Python-first → Rust ports)

(Refines an earlier wheelhouse-only idea per Felipe's 2026-06-05 note.) Rather than only feeding the build an offline wheelhouse, we **package the build tools themselves as real Debian build-dependency packages** — and use that graph as a migration instrument:

- **Start with the Python toolchain** — `fontmake`, `gftools`, `python3-ufo2ft`, `python3-fonttools`, `python3-glyphslib`, `python3-cu2qu`, `compreffor`, … — as `.deb`s in our repo,

pinned to the versions the manifests record.

- **Gradually replace each with its Rust port** as they mature: `fontc` (already), the `gftools-builder3` orchestrator, and future Rust reimplementations. Swapping a font package's Build-Depends from a Python tool `.deb` to a Rust one is a **measurable migration step**.
- **Why this beats wheelhouse-only:** (1) Debian-idiomatic — apt resolves real Build-Depends in the chroot; (2) **the `Build-Depends` graph becomes the Python→Rust burn-down** — the set of Python tool packages still required *is* the M5 blocker list, now machine-queryable across the whole library; (3) one `fontmake` `.deb` serves every family that needs it.
- **The repo therefore ships two package classes: (a) build-tool packages** (`fontmake`, `gftools`, `fontc`, `gftools-builder3`, …) and **(b) font packages** (`fonts-gf-*`).
- **Wheelhouse as a bootstrap/bridge, not the destination:** for pins not yet `.deb`-packaged, keep an offline `pip install --no-index --find-links=<wheelhouse>` step so the build stays reproducible while the `.deb` toolchain is filled in incrementally. The wheelhouse **shrinks toward empty** as packaging proceeds — itself a burn-down signal. `fontc` ships as a small `.deb` from a pinned Rust release from the start.
- **Tool bumps** (a new `fontc`, a `gftools` revision, a Rust port replacing a Python tool) → targeted **mass rebuild** of dependent font packages; the ledger (§7) flags regressions (milestone **M6 — latest-fontc currency**).

6. The apt repository (hosting & signing)

- **Repo manager:** **`aptly`** (snapshots = reproducible repository states; good for a 2,000-package living mirror + CI publishing). `reprepro` is the simpler fallback if snapshots aren't needed.
- **Signing:** a dedicated GPG key signs the `Release` file; we publish the public key and an onboarding snippet: `deb [signed-by=/usr/share/keyrings/gf.gpg] https://<host>/gf <suite> main`.
- **Layout:** standard `dists/` + `pool/`; `Architecture: all` only → one component, no per-arch pool sprawl.
- **Hosting:** a static object store / web host we control (aptly can publish to S3-compatible storage). Self-hosted, our responsibility.
- **Suites:** `stable` (verified set) and `testing` (freshly built, pending verification) so the proof-ledger **gates promotion**.

7. Build farm & the proof ledger

- **`sbuild` + `schroot`** (or `pbuilder`) clean-chroot builds — the verification core. One chroot base; per-build minimal install of declared Build-Depends.
- **Orchestrated by `gflib-build`** (reuse its scheduler, parallelism caps, cohort reuse). Per the virtiofs guidance: keep parallelism modest (≤2–3 heavy builds) and drop caches during heavy I/O.
- **The proof ledger:** every family's `sbuild` result (pass/fail + failure cause, reusing the manifest's taxonomy) is recorded. **A failure is a manifest gap**, routed back into the build-fix-provenance loop (missing `system_package`, missing `pre_build`, …). Green across

the library ⇒ the manifests are collectively proven. This ledger is the headline artifact — dashboard-surfaced alongside the M-ladder.

8. Maintenance model (self-maintained, automated)

- **Tracking upstream:** `debian/watch` + the existing "scan recent upstream commits for new/updated families" workflow detect GF/upstream changes → bump changelog (new commit in the version) → rebuild only changed families → republish. The repo becomes a **living, provenance-stamped mirror** of GF.
- **Toolchain bumps:** §5 — targeted mass rebuilds; the ledger flags regressions (M6).
- **No security-update treadmill:** fonts are data; the attack surface is the build toolchain, not the shipped fonts, so ongoing burden is dominated by *rebuild-on-change*, which is automated.

9. Staging

- **Stage 0 — one family, end to end.** Generate `debian/`, `sbuild` in a clean chroot, publish to a local `aptly`, `apt install` on a test box, confirm the font renders. Proves the whole pipeline. *(Pick a no-`pre_build`, OFL, static-TTF family first.)*
- **Stage 1 — a slice (~30–50 families).** Exercise the generator across the hard cases: VFs, OTF, `pre_build` (C), `system_packages` (B, e.g. a Cairo-dependent family), license variety (Apache/UFL). Harden the generator, the tool packages, and the wheelhouse bridge.
- **Stage 2 — full library.** Drive all families through generate → `sbuild` → publish; build the proof ledger; iterate failures back into the manifests until green (or documented-unbuildable). The long pole — compute-bound.
- **Stage 3 — maintenance automation.** CI on upstream/toolchain change; promotion `testing` → `stable` gated by the ledger.
- **Parallel track — the Python→Rust tool packaging (§5):** package the Python toolchain early (unblocks Stage 1–2), then replace tools with Rust ports as they land, watching the Build-Depends burn-down.

10. Effort, resources, risks

- **Effort:** Stage 0 ~1 session; Stage 1 a few; Stage 2 several (compute-bound, like the existing full-library build). New engineering = the `debian/` generator + the tool packages + the `aptly`/`sbuild` glue; per-family work is automated.
- **Disk:** source packages + built `.deb`s + chroots + wheelhouses are large. **Honor the policy:** `df -h` before big ops, refuse < 15 GB free, shallow where possible.
- **Risks:** (1) toolchain reproducibility — mitigated by real `.deb` build-deps + the offline wheelhouse bridge; (2) build flakiness at scale — clean chroots + the ledger; (3) file/namespace clashes with official font packages — the `fonts-gf-*` + `gf-<family>` namespace; (4) hosting cost/bandwidth for a 2,000-package repo — sizing needed; (5) maintenance scale — full automation + `Architecture: all` (no per-arch matrix).

11. Relationship to existing work

- **`build-fix-provenance.md`** — this plan is its **verification + distribution layer**: the Debian build *consumes* the manifest and *proves* it in a clean room. The manifest stays the single source of truth; `debian/` is generated from it.
- **`gflib-build`** — host the generator (`--export deb`) and the `sbuild` orchestration here; reuse discovery/cohorts/scheduler/ledger.
- **North-star milestones** — two independent burn-downs fall out for free: (a) a font package that builds with a **pure-`fontc` + `builder3`, empty-`pre_build`, no-wheelhouse** recipe is by construction at **L5/L6**; (b) the **`Build-Depends` graph's remaining Python tool packages are the M5 blocker list**. The Debian ledger doubles as an M4/M5/M6 burn-down with independent, clean-room evidence.
- **Debian methods** (provenance doc §9) — DEP-5, DEP-3, `debian/watch`, `Build-Depends`, `.buildinfo` / reproducible-builds — are all **used for real** here, not merely borrowed as schemas.

Drafted by an AI agent (Claude Opus 4.8) under the guidance of @felipesanches.